DAY 32 | HANDS-ON ACTIVITIES

LLM Evaluation and Quality Gates: Problem Statements

Eight problems that turn evaluation into automatic quality checks. All run offline; with a key, model-graded checks become meaningful. Offline results show how the code works, not real model quality. Prices are placeholders.

Prepared by **Ahmed** | linkedin.com/in/ahmed-al-imaad

How to work: read the problem, write the steps in plain English first, then write the code. Test with the sample run, then try your own inputs, including wrong ones.

Problem 1: Golden Set v1

Level: Warm-up | **Concepts:** dataset curation

Write 12 golden items in JSONL with id, question, expected, relevant chunk ids, tags, must_include and must_not_include. Save as golden_v1.jsonl.

Sample run / expected result

```
{"id": "leave-003", "question": "When does carried over leave expire?", "must_include": ["31
```

Hints

- must_not_include catches common wrong answers.

Problem 2: Deterministic Tests

Level: Easy | **Concepts:** PyTest parametrize

Write a parametrized PyTest that checks must_include and must_not_include for every golden item, with item ids shown in the test names.

Sample run / expected result

```
12 passed in 0.8s
```

Hints

- ids=lambda i: i["id"] gives readable test names.

Problem 3: Catch a Regression

Level: Medium | **Concepts:** prompt and config versions

A teammate changes k from 3 to 1 "to save cost" in config v2. Run the golden tests for v1 and v2 and report which items regressed.

Sample run / expected result

```
v1: 8/12 passed (offline)
v2: 6/12 passed
Regressed: leave-003, travel-003
```

Hints

- Compare per item, not only totals.

Problem 4: Tool-call Fidelity NEW (v4)

Level: Medium | **Concepts:** tool selection, JSON Schema, exact arguments

Write 8 cases for the SmartDesk assistant (refund, plan change, ticket lookup, and two where no tool should be called). Test the tool name, schema validity and exact argument values.

Sample run / expected result

```
8 cases: tool 8/8 | schema 7/7 | exact args 6/7
FAIL: 'Refund 1,200 on ORD-9' -> amount 1 (comma parsing)
```

Hints

- A wrong amount is worse than no call. Test values, not just names.

Problem 5: Context Precision, Recall and Citations

Level: Medium | **Concepts:** retrieval metrics, citations

Compute precision, recall and average precision for k = 1, 3 and 5, then check citation accuracy: is every cited chunk retrieved and relevant?

Sample run / expected result

```
k=1: precision 0.58 recall 0.58
k=5: precision 0.18 recall 0.92
citation accuracy: 0.83
```

Hints

- More chunks usually raise recall and lower precision.

Problem 6: BLEU from Scratch

Level: Medium | **Concepts:** n-grams, brevity penalty

Implement BLEU (1 to 4-grams, brevity penalty). Score an exact copy, a correct paraphrase and a wrong answer that reuses words. Explain the result.

Sample run / expected result

```
copy 1.00 | paraphrase 0.11 | wrong but similar words 0.63
```

Hints

- Add 1 smoothing so a missing 4-gram does not make the score zero.

Problem 7: Cost-aware Quality Gate NEW (v4)

Level: Challenging | **Concepts:** thresholds, tokens, cost, exit codes

Extend quality_gate.py so it fails when recall < 0.85, pass rate drops more than 0.05 versus the v1 baseline, or estimated cost per query rises more than 20% versus the baseline. Show v1 pass, v2 fail on quality and v3 (k = 8) fail on cost.

Sample run / expected result

```
v1: PASS
v2: FAIL recall 0.58 < 0.85
v3: FAIL cost per query up 54% vs baseline -> exit 1
```

Hints

- sys.exit(1) makes the CI step fail.
- Label prices as placeholders in the report.

Problem 8: Eval-first Feature with Codex NEW (v4)

Level: Stretch | **Concepts:** eval-first development, AGENTS.md

Write golden items requiring a citation like [c7] in every answer. Run them and see them fail. Then ask Codex to add citations to the RAG answer, rerun the evals and the gate, and record the diff summary.

Sample run / expected result

```
before: citation tests 0/12
after Codex change: 12/12, gate PASS
```

Hints

- Add the eval command to AGENTS.md so Codex runs it after each change.